AN0290
Application Note

AT32F405 High-speed USB Magnetic Switch Keyboard Application

Introduction

Magnetic switch keyboard is a keyboard with magnetic switches that use magnetic fields to detect key presses.

Compared with traditional mechanical switches or membrane keyboards, magnetic switch keyboards have the following features:

- Contactless sensing: Utilize Hall effect sensors to detect the movement of magnets attached to the key stems instead of traditional metal contacts, reducing physical wear and failures.
- Durability: Magnetic switch keyboards can maintain consistent performance over time and last millions of keypresses since they do not rely on physical contact.
- Responsiveness: Hall effect sensors are used to provide rapid and stable signal changes and in combination with the high-speed scanning capability of MCUs to achieve low-latency key actuation response.
- Adjustable actuation: Key travel, trigger point and rebound point can be adjusted through hardware or software, providing a customizable user experience.

Magnetic switch keyboards are widely used in scenarios such as gaming, high-end input devices and professional control panels. Based on MCU system, it achieves high integration of keyboard scanning, USB HID communication and LED backlight control.

This application note introduces a demo of AT32 MCU-based highs-speed USB magnetic switch keyboard, featuring an 8 KHz polling rate, 16 KHz scanning rate and keyboard sensitivity fine-tuning.

Application products:

Part number	AT32F405

Contents

1 Overview 5

1.1 Hardware requirements 5

1.2 Software requirements 6

2 Software implementation 8

2.1 Keyboard scanning 8

2.1.1 Scanning process 9

2.1.2 Code execution 10

2.2 Key state 10

2.2.1 Key state machine 10

2.3 USB HID KEY report 12

2.4 USB Suspend state power consumption management 13

2.5 RGB LED control 14

3 Revision history 15

List of tables

Table 1. Key variables.....11

Table 2. Document revision history 15

List of figures

Figure 1 Overall architecture..... 5
Figure 2 AT32-Magnetic-Gamings-Keyboard development board..... 5
Figure 3 Online upgrade through IAP host PC 7
Figure 4 Analog multiplexer connection settings 8
Figure 5 Keyboard scanning flowchart..... 9
Figure 6 Key state machine update diagram.....11

1 Overview

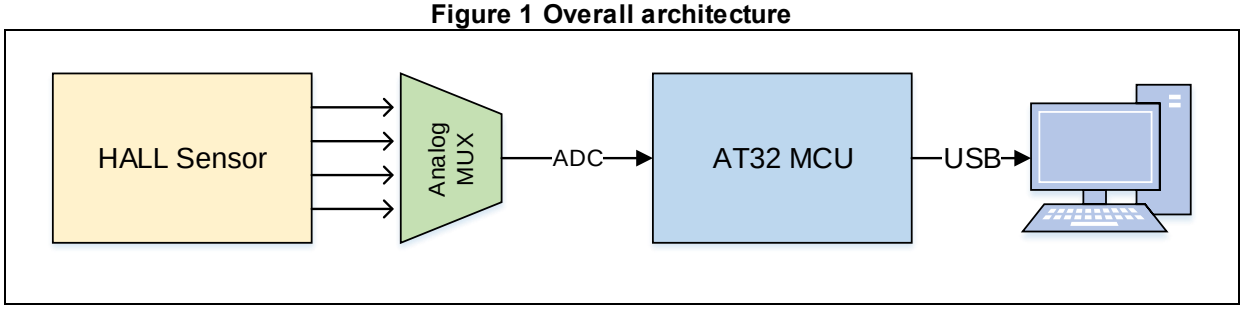
This document introduces a demo of AT32 MCU-based high-speed USB magnetic switch keyboard.

The content includes system architecture, hardware design, software architecture (IAP+APP), keyboard scanning (ADC+DMA+analog multiplexer), USB communication and the implementation of WS2812 RGB LED control, providing references for the development of magnetic switch keyboards.

Main features:

- 8 KHz polling rate
- 16 KHz scanning rate
- Hall effect sensor for magnetic switch key input
- 68-key design
- ADC+DMA+analog multiplexer for high-efficiency keyboard scanning
- Optional Kalman 1/ Kalman 2 / EMA (Exponential Moving Average) for data filtering
- Adjustable trigger sensitivity
- WS2812 RGB LED display
- USB High-Speed interface
- IAP+APP architecture, supporting online upgrade

Figure 1 Overall architecture



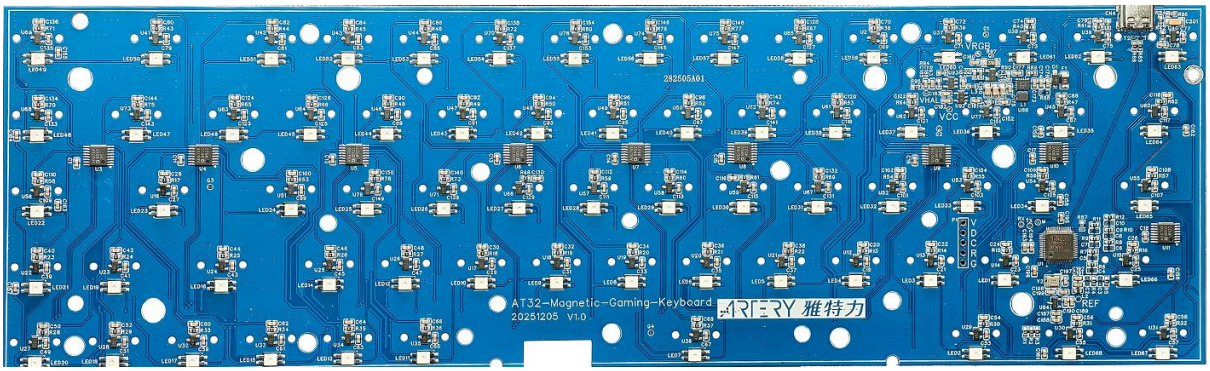
1.1 Hardware requirements

This demo is based on AT32-Magnetic-Gamings-Keyboard development board to implement magnetic switch board functions through AT32 MCU. Below devices and parts are required:

- AT32 MCU (main control chip)
- Hall effect sensor array (for magnetic switch sensing)
- Analog multiplexer (8:1, for expanding ADC input channels)
- WS2812 addressable RGB LED
- USB High-Speed interface

For detailed circuits, refer to *SCH_AT32-Magnetic-Gaming-Keyboard.Vxx.pdf*.

Figure 2 AT32-Magnetic-Gamings-Keyboard development board



1.2 Software requirements

This demo utilizes IAP+APP architecture. The path is as follows:

- **IAP bootloader**
  Path: utilities\bootloader\mdk_v5
  Function: firmware upgrade and program jump
  Flash address: 0x08000000~0x080006000 (24 KB)
- **Keyboard APP**
  Path: utilities\keyboard\mdk_v5
  Function: keyboard scanning, USB communication and RGB LED control display
  Flash address: 0x080006000~0x08040000 (232 KB)

Keyboard APP project structure:

```
keyboard/
├─ src/
│  ├─ main.c // system entry
│  ├─ keyboard.c // keyboard scanning + state machine core
│  ├─ keyboard_adc.c // keyboard ADC
│  ├─ keyboard_filter.c // keyboard filtering algorithm
│  ├─ keyboard_power.c // keyboard suspend status power consumption handling
│  ├─ keyboard_usb.c // keyboard USB interface
│  ├─ keymap.c // physical key index and HID keycode mapping
│  ├─ keyboard_class.c // USB HID keyboard-related processing
│  ├─ keyboard_desc.c // HID descriptor
│  ├─ hid_iap_user.c // HID IAP-related processing
│  └─ rgb_led.c // WS2812 LED driver
```

Operation procedures:

1. Program Bootloader
2. Program Keyboard APP
3. Power on and then Bootloader jumps to APP

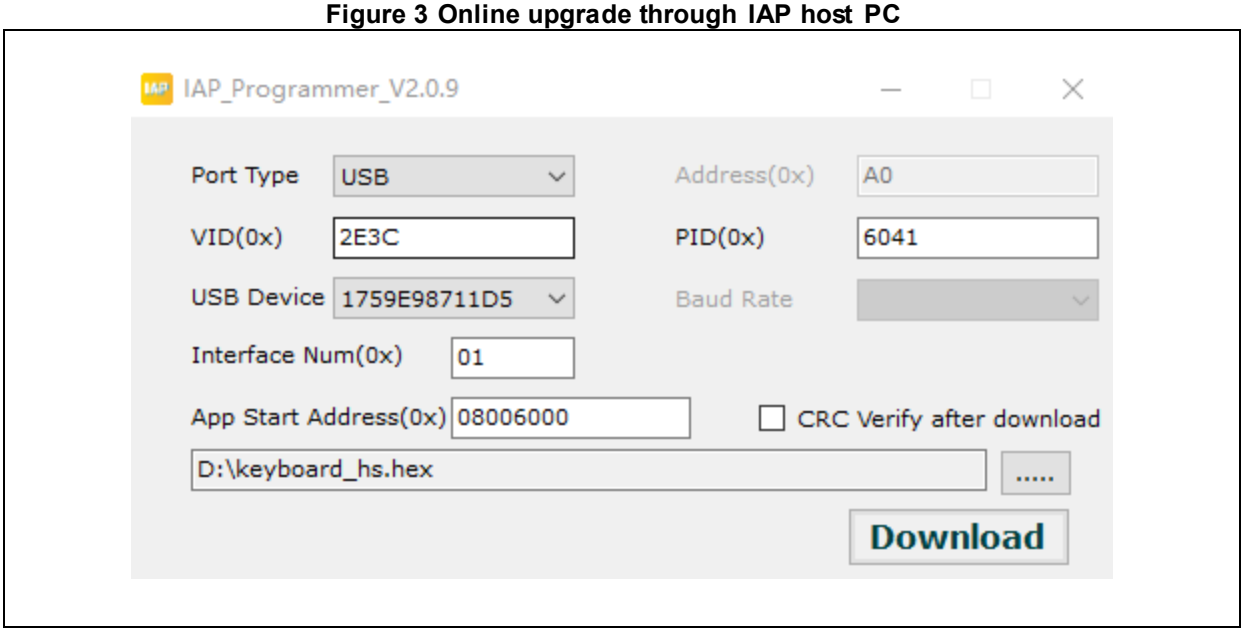
Online upgrade can be completed through the Tool/IAP_Programmer.

IAP host PC operation:

- Open “IAP Programmer”
- Go to “Port type” and select “USB”
- Go to “Interface Num” and select “0x01”
- Input “0x6041” to “PID”
- Select the USB Device and the corresponding hex file
- Click on “Download”

The flowchart is shown below.

Figure 3 Online upgrade through IAP host PC



Note: All projects are built around Keil 5. If users want to use in other compiling environment, refer to AT32Fxx_Firmware_Library_V2.x.x\project\at_start_f4xx\templates (such asIAR6/7, Keil 4/5) for a simple modification.

2 Software implementation

This section introduces how to implement the magnetic switch keyboard demo, including keyboard scanning, key state determination, RGB LED control and USB HID communication.

Software execution process after the system power-on:

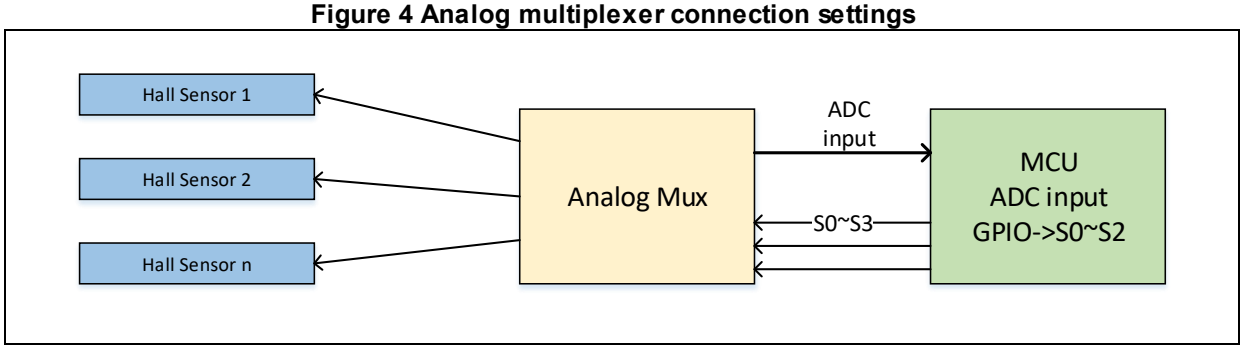
```
main()
├─ system_clock_config() // clock initialization
├─ wk_timebase_init() // delay function initialization
├─ usbd_init()          //USB initialization
├─ rgb_led_init()       //ws2812 RGB LED initialization
├─ keyboard_init()      // keyboard scanning (ADC,DMA) initialization and calibration
└─ while(1)
    └─ keyboard_task() // keyboard scanning + algorithm processing + report
    └─ rgb_led_task()  //RGB LED processing
```

2.1 Keyboard scanning

The magnetic switch keyboard utilizes **ADC + DMA + analog multiplexer** to implement high-efficient keyboard scanning. Each key has a dedicated Hall effect sensor. Its output voltage varies linearly or quasi-linearly with magnet position (key travel), enabling continuous and quantifiable displacement sensing.

Due to the large number of keyboard keys and the limited number of available on-chip ADC input channels, analog multiplexers (8:1) are introduced in hardware design for channel extension of Hall effect sensor signals. The output signals of multiple Hall effect sensors are connected to the input terminals of the analog multiplexers. By controlling the select signals of the multiplexers, the analog voltage of the designated channel is output to the MCU ADC input, thereby achieving polled sampling of multiple keys.

Figure 4 Analog multiplexer connection settings



In terms of software implementation, the system sequentially switches the analog multiplexer channels according to a predefined scanning sequence, and triggers ADC conversion after each channel switching has stabilized. The ADC sampling results are automatically transferred to a memory buffer via DMA. The double-buffer structure is used, which allows ADC conversion and key parsing to be performed simultaneously, thereby improving system efficiency.

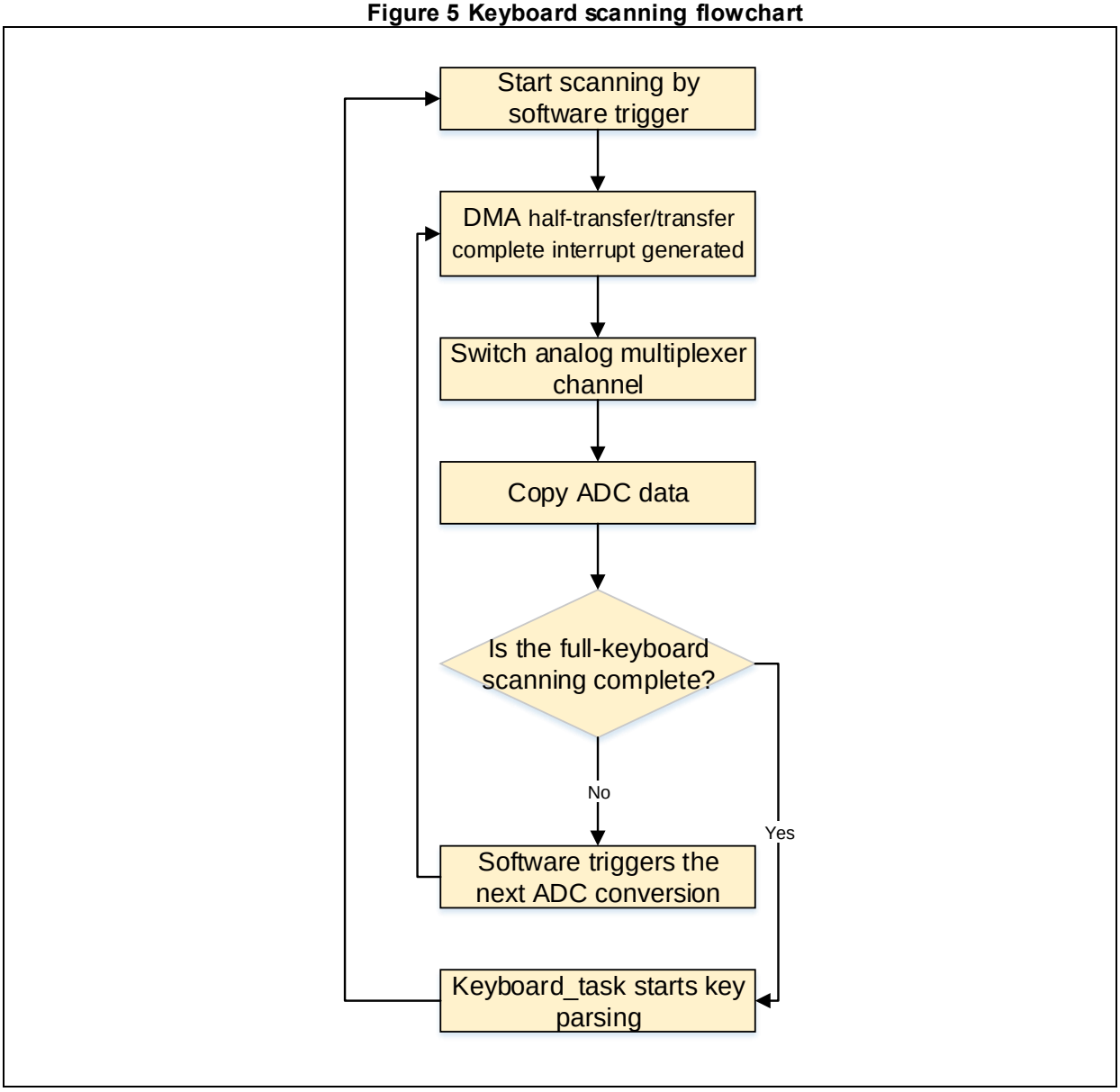
2.1.1 Scanning process

The keyboard contains a total of 68 keys, and each key corresponds to one ADC sampling value. Each ADC channel is extended via an 8:1 analog multiplexer to connect external Hall effect sensors. During full-keyboard scanning, after the ADC completes each conversion sequence, it must switch the analog multiplexer to select the next Hall effect sensor for sampling.

Sampling procedures:

- Software triggers ADC to start a keyboard scanning;
- DMA half-transfer/transfer complete interrupt is generated;
- MCU controls GPIO to switch the analog multiplexer channel;
- After the channel stabilizes, start ADC for the next conversion;
- After a full-keyboard scanning is completed, wait for the CPU to start the next scanning.

Figure 5 Keyboard scanning flowchart



2.1.2 Code execution

Code execution during keyboard scanning:

```
adc_ordinary_software_trigger_enable() // software triggers ADC conversion in sequence mode
├─ KEY_ADC_DMA_IRQ_HANDLER() // one ADC sequence mode conversion complete interrupt handler
    └─ key_matrix_cs_switch() // switch to the next multiplexer channel
    └─ adc_ordinary_software_trigger_enable() // trigger the next ADC conversion
    └─ copy_data // copy the current completed data
└─ keyboard_task // key state parsing
```

2.2 Key state

Since magnetic switch keyboard outputs analog signals that may be subjected to power supply ripple, ambient noise and component discreteness in practical applications, ADC sampling data needs to be processed to reliably determine key states and further generate valid keycode information.

This demo supports filtering options including EMA (Exponential Moving Average), Kalman 1 and Kalman 2. These algorithms process raw ADC samples to achieve trade-offs between responsiveness and stability.

Kalman 1 employs a one-dimensional Kalman filter algorithm. It models system noise and measurement noise using fixed parameters, and dynamically adjusts the filtering gain based on estimation error. This achieves high stability while significantly reducing filtering latency, making it ideal for gaming keyboards and Rapid Trigger applications that demand high real-time performance.

Kalman 2 is an advanced filtering mode. On the basis of Kalman 1, it introduces higher-precision or adaptive strategies that dynamically adjust filtering behavior based on key movement state. It achieves a superior balance between fast response and static stability, delivering lower latency and better tracking performance. Therefore, it is suitable for high-end analog keyboard applications with the highest requirements for performance and user experience.

The system determines key states based on key travel and relative movements. Key sensitivity is adjustable via configurable actuation displacement thresholds, meeting requirements for trigger precision and typing experience in different application scenarios.

2.2.1 Key state machine

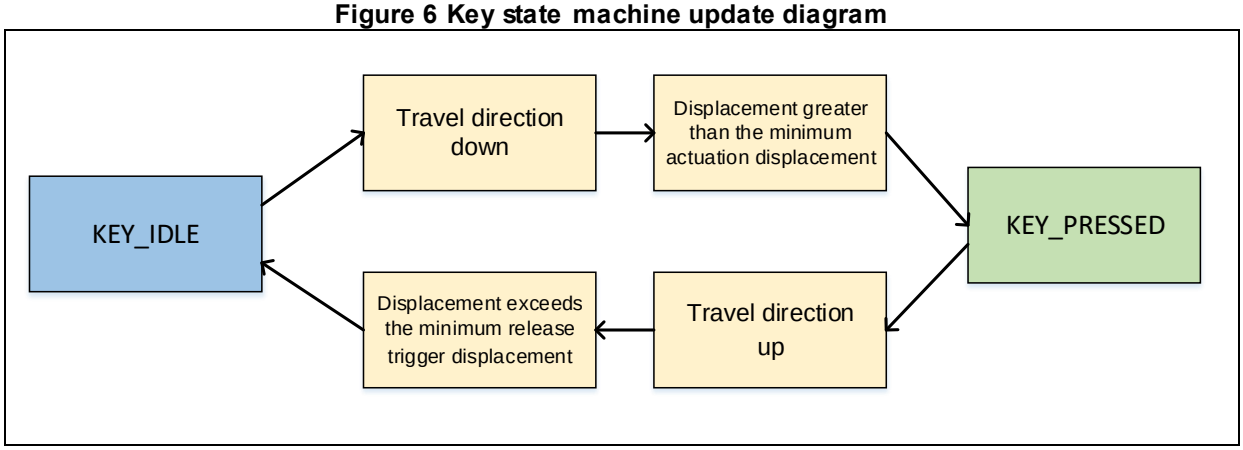
Unlike traditional mechanical keyboards with only “on/off” contact states, magnetic switch keyboards capture continuous displacement of keys using Hall effect sensors. As such, the key actuation is inherently a continuous analog variation process.

Using fixed actuation / release travel for key state determination will result in the following issues:

- Slow or delayed release during rapid consecutive presses
- Hysteresis in mid-travel operation
- Fails to fully exploit the continuous travel advantage of magnetic switches

Therefore, this design introduces a simple Rapid Trigger (RT) key state machine to achieve dynamic actuation and release by parsing the key travel direction and relative displacement changes. The key state machine update diagram is shown below:

Figure 6 Key state machine update diagram



2.2.1.1 State machine design

The state machine relies on the following three points, instead of the absolute ADC threshold:

- Travel direction determination (dv)
- Single travel peak record (adc_max_value/ adc_min_value)
- Relative displacement variation (rt_press_delta/ rt_release_delta)

To summarize the core determination principle:

When a sufficiently clear displacement trend is detected in either key movement direction, a state switch is triggered.

Key sensitivity can be adjusted by modifying the relative displacement variations (rt_press_delta/ rt_release_delta).

2.2.1.2 Key variables

Critical variables for key state determination are listed below.

Table 1. Key variables

Variable	Description
adc_filter_value	Current displacement value of the key after ADC sampling + EMA filtering
dv	Displacement change, used to determine the direction of travel
DV_DEADZONE	Travel direction dead zone, used to suppress small jitters
adc_max_value	Maximum displacement value during the current key stroke
adc_min_value	Minimum displacement value during the current key stroke
rt_press_delta	Minimum relative displacement required for actuation
rt_release_delta	Minimum relative displacement required for release

2.2.1.3 State logic

The state machine consists of two basic states.

- **RT_KEY_IDLE**
  The key is in the released or idle state, and the system monitors for a pressing trend.
- **RT_KEY_PRESSED**
  The key is determined as pressed, and the system monitors for a rebound trend.

1. **Key actuation determination logic**

   Maximum travel record

```
if (dv > DV_DEADZONE  && value > k->adc_max_value)
    k->adc_max_value = value;
```

   When a distinct downward pressing trend is detected, continuously update the maximum displacement for the current stroke, providing a reference baseline for subsequent relative displacement judgment.

Actuation conditions

```
if (dv <= DV_DEADZONE  &&
      k->adc_max_value - value > k->rt_press_delta)
```

Actuation requires both conditions to be met:

- Current displacement change is stabilized (entered the direction dead zone)
- The retracement amplitude relative to the current maximum displacement exceeds rt_press_delta

This design effectively prevents:

- False trigger caused by minor jitter
- Incorrect judgment when a clear travel trend has not yet formed

Once both conditions are met, the key state switches to RT_KEY_PRESSED, and the corresponding HID key press event is generated.

2. **Key release determination logic**

Minimum travel record

```
if (dv < DV_DEADZONE  && value < k->adc_min_value)
      k->adc_min_value = value;
```

When an upward rebound trend in key displacement is detected, record the minimum displacement value during the rebound process.

Release conditions

```
if (dv >= DV_DEADZONE  &&
      (value - k->adc_min_value > k->rt_release_delta))
```

Conditions of release:

- The displacement trend re-enters a stable state or a downward direction
- The retracement amplitude relative to the minimum displacement exceeds rt_release_delta

This design effectively prevents:

This mechanism allows the key to release before fully rebounding to its initial position, significantly improving response speed for rapid consecutive presses and pre-release re-actuation.

3. **Auto learn**

When the SUPPORT_AUTO_LEARN_RT is enabled, the system can perform auto learning and updates of RT parameters after a full press-release process, adapting to variations between different switches or magnets.

To ensure stable learning results, it is recommended to perform updates only when the travel amplitude reaches the preset minimum value.

2.3 USB HID KEY report

The magnetic switch keyboard system communicates with the host via USB High-Speed HID (Human Interface Device) protocol to implement standard keyboard input functionality and works without any additional drivers.

2.3.1.1 KEY report interface

USB HID keyboard report is transmitted via a unified interface function.

```
uint8_t hid_keyboard_report(uint8_t *data, uint8_t len)
{
   if(p_key->is_suspend) {
      clear_keyboard_suspend();
      usbd_remote_wakeup(&otg_core_struct.dev);
      return 0;
```

```
    } else {
        return (uint8_t)usb_keyboard_class_send_report(&otg_core_struct.dev, (uint8_t *)data, 8);
    }
}
```

This interface is used for handling the following two types of scenarios:

1. Keyboard data reporting
2. Remote Wakeup when USB is in “Suspend” state

- Normal data reporting
  usb_keyboard_class_send_report(&otg_core_struct.dev, (uint8_t *)data, 8);
  Report length is fixed at 8 bytes
- Suspend remote wakeup
  Key press event is detected when the keyboard is in “Suspend” state
  1. Clear the local Suspend flag
  2. Send remote wakeup signal to the host via usbd_remote_wakeup()

2.3.1.2 Keycode management

The keyboard runs in 6KRO (6 key Rollover) mode, which means that up to six ordinary keys can be reported together.

Add keys:

```
static void keyboard_add_key(uint8_t keycode)
```

This function writes keycodes into keycode[] array in sequence, and it automatically wraps around when exceeding 6 keys.

Clear keys:

```
static void keyboard_clear_key(uint8_t keycode)
```

Loop through the 6 key slots and set to 0 if a matching keycode is found.

2.4 USB Suspend state power consumption management

When the USB bus enters Suspend state, the system power consumption is reduced by lowering MCU frequency and disable RGB display.

- **Lower MCU frequency**
  After the device enters Suspend state, set the system frequency to 48 MHz to reduce power consumption.

```
if(p_key->is_lowpower == FALSE && usbd_is_keyboard_suspend()== TRUE) {
    /* enter low power */
    suspend_enter_lower_power();
}
```

- **Disable RGB display**
  After the device enters Suspend state, RGB display is disabled to reduce power consumption.

```
if(usbd_is_keyboard_suspend()) {
    if(rgb_led_suspend == 0) {
        rgb_led_suspend = 1;
        LED_RGB_OFF();
    }
    return;
}
```

2.5 RGB LED control

This magnetic switch keyboard demo integrates WS2812 addressable RGB LED to realize simple breathing and marquee light effects

WS2812 transmits data via a single-wire serial interface and has strict timing requirements; therefore, signal accuracy and stability must be guaranteed for software implementation.

This demo offers the below two typical implementation methods.

Control mode	Macro	Features
SPI+DMA	WS2812_CONTROL_USE_SPI	Simulate WS2812 waveform using SPI and transmit data automatically via DMA
TMR(PWM)+DMA	WS2812_CONTROL_USE_TMR	Use a timer to output PWM waveforms, and dynamically update the compare value via DMA.

Both control modes can stably drive a large number of RGB LEDs without consuming excessive CPU resources.

This demo provides basic display modes in terms of lighting effect design, including:

- Breathing light
- Flow light

Developers can link LED display to key status, operating mode, or host commands as needed, achieving richer and more personalized lighting effects.

3 Revision history

Table 2. Document revision history

Date	Version	Revision note
2026.01.22	2.0.0	Initial release.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services, and ARTERY assumes no liability whatsoever relating to the choice, selection or use of the ARTERY products and services described herein.

No license, express or implied, to any intellectual property rights is granted under this document. If any part of this document deals with any third party products or services, it shall not be deemed a license granted by ARTERY for the use of such third party products or services, or any intellectual property contained therein, or considered as a warranty regarding the use in any manner of such third party products or services or any intellectual property contained therein.

Unless otherwise specified in ARTERY's terms and conditions of sale, ARTERY provides no warranties, express or implied, regarding the use and/or sale of ARTERY products, including but not limited to any implied warranties of merchantability, fitness for a particular purpose (and their equivalents under the laws of any jurisdiction), or infringement on any patent, copyright or other intellectual property right.

Purchasers hereby agree that ARTERY's products are not designed or authorized for use in: (A) any application with special requirements of safety such as life support and active implantable device, or system with functional safety requirements; (B) any aircraft application; (C) any aerospace application or environment; (D) any weapon application, and/or (E) or other uses where the failure of the device or product could result in personal injury, death, property damage. Purchasers' unauthorized use of them in the aforementioned applications, even if with a written notice, is solely at purchasers' risk, and Purchasers are solely responsible for meeting all legal and regulatory requirements in such use.

Resale of ARTERY products with provisions different from the statements and/or technical characteristics stated in this document shall immediately void any warranty grant by ARTERY for ARTERY's products or services described herein and shall not create or expand any liability of ARTERY in any manner whatsoever.

© 2026 Artery Technology - All rights reserved